

Semiconductor Manufacturing Defect Prediction & Quality Intelligence

An End-to-End Machine Learning System Report

Advanced AI/ML Assignment

September 5, 2026

Project Type: Applied Machine Learning Systems – Predictive Quality Analytics

Domain: Semiconductor Wafer Fabrication

Deliverables: Synthetic Dataset, Trained Model Pipeline, REST API, Interactive Dashboard

Abstract

Semiconductor fabrication is one of the world's most precision-sensitive manufacturing processes, where sub-millimetre fluctuations in chamber temperature, pressure or equipment vibration can result in costly, hard-to-detect wafer defects. This report covers the complete design, implementation, and deployment of a machine learning system built to predict rare manufacturing defects using process telemetry. A domain-realistic synthetic dataset of 25,000 process records was engineered with correlated anomaly patterns, sensor dropout simulation and a deliberately tuned class imbalance of 8.33% to mirror real fabrication defect rates. After a context-aware exploratory data analysis and feature engineering (including equipment-stress and maintenance-risk interaction features), a Logistic Regression model trained in a leakage-safe SMOTE pipeline was compared with a tuned XGBoost ensemble. Logistic Regression had the best Precision-Recall AUC (0.3994) and significantly higher recall (0.5852 vs. 0.2545) making it the better model for a rare-event detection use case where missing defects is much more expensive than false alarms. The final pipeline was serialized and served via a FastAPI microservice with strict Pydantic input validation, consumed by a real-time React and TypeScript dashboard styled as a "Premium Control Room" operator interface. To summarize, the system provides a reproducible, leakage-aware and production-ready pathway to rare-event predictive quality analytics in manufacturing.

Contents

1	Introduction	4
1.1	Background	4
1.2	Motivation	4
1.3	Problem Context	4
1.4	Problem Statement	4
1.5	Objectives	5
1.6	Scope	5
1.7	Report Organization	5
2	Assignment-Oriented Methodology	5
3	Milestone 1 — Scenario Selection & Synthetic Data Engineering	6
3.1	Industry Scenario	6
3.2	Predictive Task	6
3.3	Synthetic Dataset Design	6
3.4	Target Construction	7
3.5	Class Imbalance	7
3.6	Missing Values	7
3.7	Correlated Anomalies	7
3.8	Reproducibility	7
4	Milestone 2 — Context-Aware EDA & Feature Engineering	8
4.1	Data Quality Analysis	8
4.2	Missing-Value Analysis	8
4.3	Outlier and Anomaly Analysis	8

4.4	Class-Imbalance Analysis	9
4.5	Data Cleaning	10
4.6	Sampling / Imbalance Strategy	10
4.7	Feature Engineering	10
4.8	Interaction Features	10
4.9	Temporal / Historical Features	11
4.10	Scaling and Encoding	11
4.11	Exploratory Data Analysis	11
4.12	Key EDA Findings	11
5	Milestone 3 — Algorithmic Modeling & Performance Rigor	12
5.1	Dataset Splitting	12
5.2	Baseline Model	12
5.3	Advanced Model	12
5.4	Model Training	12
5.5	Hyperparameter Optimization	12
5.6	Evaluation Metrics	12
5.7	Why Accuracy Alone Is Insufficient	13
5.8	Model Comparison	13
5.9	Final Model Selection	14
5.10	Model Interpretability	14
5.11	Model Serialization	15
6	Milestone 4 — Backend API Construction & Automated Testing	15
6.1	Backend Architecture	15
6.2	REST API	15
6.3	Prediction Endpoint	15
6.4	Input Schema	16
6.5	JSON Validation	16
6.6	Model Loading	16
6.7	Automated Testing	16
7	Milestone 5 — Interactive Front-End Dashboard Deployment	17
7.1	Dashboard Architecture	17
7.2	Design Goals	18
7.3	Dashboard Features	18
7.4	Final User Interface (UI)	18
7.5	Prediction Result Visualization	19
7.6	Responsiveness and Accessibility	19
8	Results & Discussion	19
9	Limitations	20
10	Future Scope	20
11	Conclusion	21
12	References	21

List of Figures

1	High-level system architecture diagram (data generation → preprocessing → model → API → dashboard)	6
2	Missing-value heatmap across the three affected sensor columns	8
3	Distributions of vibration level and chamber pressure, segmented by defect label	9
4	Class distribution bar chart (Normal vs. Defect)	10
5	Feature correlation heatmap across engineered and raw features	11
6	Precision-Recall curve comparison (Logistic Regression vs. XGBoost)	13
7	Confusion matrix for the selected Logistic Regression model	14
8	Logistic Regression coefficient importance chart	15
9	Terminal output of the pytest test suite run (all tests passing)	17
10	Interactive Swagger/OpenAPI documentation for the /predict endpoint	17
11	Interactive Telemetry Dashboard — Initial State	18
12	Prediction Results demonstrating a High Risk Threshold	19

List of Tables

1	Model comparison on the held-out test set.	13
2	API automated testing results (6/6 tests passing).	16

1 Introduction

1.1 Background

Each of the hundreds of sequential process steps involved in semiconductor manufacturing—deposition, etching, lithography, doping, and polishing—must be carried out within incredibly precise physical tolerances. Thousands of sensors that continuously stream telemetry, including chamber temperature, chamber pressure, gas flow rate, and equipment vibration, are installed in modern fabrication facilities (fabs). Even slight variations in these readings can result in wafer-level flaws that are only found much later in the production line, at which point the cost of rework or scrap is much higher than if the defect had been discovered earlier. This is especially true when the flaws compound across an aging piece of equipment.

1.2 Motivation

Conventional statistical process control (SPC) charts are good at identifying single-variable deviations, but they have trouble capturing multi-variable, non-linear interactions between sensors, such as the compounded risk of an aging machine running at a slightly higher pressure. As long as the rarity of true defect events (the "needle in a haystack" problem) is appropriately addressed during training and evaluation, machine learning models are well suited to learn these interaction effects directly from historical telemetry.

1.3 Problem Context

Because a single defective wafer can result in the loss of dozens of individual chips, defects in semiconductor fabrication are extremely costly despite being intrinsically rare, frequently occurring in the single-digit percentage range. Significant cost savings can be achieved by anticipating these flaws before they eventually spread. The synthetic data engineering approach used in this project was motivated by the fact that actual proprietary foundry telemetry is highly protected intellectual property and is nearly impossible to obtain for academic or exploratory work.

1.4 Problem Statement

The objective of this project is to architect a robust, production-ready machine learning system that:

1. Ingests raw process telemetry (numerical and categorical),
2. Handles severe class imbalance without leaking synthetic samples into validation data,
3. Trains and rigorously evaluates models optimized specifically for rare-event detection, and
4. Serves accurate, validated, real-time risk predictions through a deployable API and dashboard.

1.5 Objectives

- O1.** Synthesize a domain-accurate manufacturing dataset with realistic, correlated anomaly structure.
- O2.** Engineer features that capture equipment drift, maintenance history, and non-linear sensor interactions.
- O3.** Train, tune, and select a model optimized for precision-recall performance on rare events.
- O4.** Deploy the selected model behind a secure, schema-validated REST API.
- O5.** Provide plant operators with an interactive, real-time quality-intelligence dashboard.

1.6 Scope

The system focuses strictly on tabular process telemetry (sensor readings, equipment metadata, and recipe parameters) and does not extend to image-based or computer-vision-based wafer defect detection, which is treated as a separate class of problem and left for future work.

1.7 Report Organization

The remainder of this report is organized around the five milestones of the assignment: synthetic data engineering (Section 3), exploratory data analysis and feature engineering (Section 4), algorithmic modeling (Section 5), backend API construction (Section 6), and front-end dashboard deployment (Section 7), followed by consolidated results, limitations, and future scope.

2 Assignment-Oriented Methodology

The system was developed strictly according to the five required milestones outlined in the Advanced AI/ML Assignment specification, ensuring complete compliance with the provided requirements while allowing room for additional engineering rigor, such as group-aware data splitting and pipeline-safe resampling, beyond the minimum expectations. Figure 1 summarizes the overall system architecture, from raw synthetic telemetry through to the operator-facing dashboard.

System Architecture Overview

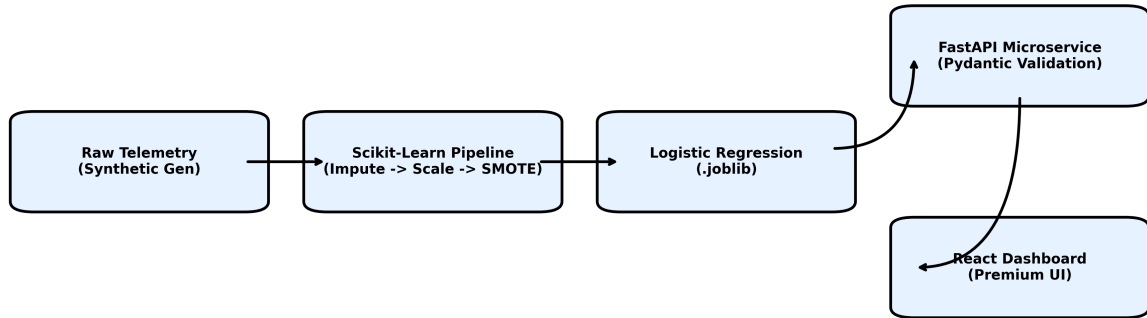


Figure 1: High-level system architecture diagram (data generation → preprocessing → model → API → dashboard)

3 Milestone 1 — Scenario Selection & Synthetic Data Engineering

3.1 Industry Scenario

Semiconductor wafer fabrication was selected as the target industry scenario due to its well-documented reliance on precise, sensor-driven process control and its naturally rare, high-cost defect events, properties that make it an excellent testbed for imbalanced-classification methodology.

3.2 Predictive Task

The core predictive task is framed as binary classification: given a wafer’s process telemetry, predict whether the wafer will exhibit a manufacturing defect (label 1) or remain within normal specification (label 0).

3.3 Synthetic Dataset Design

The dataset comprises 25,000 synthetic process records, each representing one processed wafer.

- **Numerical Features:** `chamber_temperature`, `chamber_pressure`, `vibration_level`, `equipment_age`, `gas_flow_rate`, `maintenance_days_since`, and related process telemetry fields.
- **Categorical Features:** `wafer_type`, `equipment_id`, `process_recipe`.

- **Identifier Fields:** `batch_id`, used later to enforce group-aware train/test splitting (Section 5.1) so that wafers from the same production batch never leak across the train/test boundary.

3.4 Target Construction

A synthetic risk score was generated using a modified sigmoid function applied to compounded temperature, pressure, and vibration deviations from their nominal operating baselines. The sigmoid formulation was deliberately chosen over a simple linear threshold rule because it produces a smooth, bounded probability surface—avoiding hard, unrealistic decision boundaries—while still allowing the compounded deviation terms to dominate the resulting risk in a physically plausible way. The binary defect label was then sampled using this synthetic risk score as a Bernoulli probability, preserving stochastic realism rather than deterministically labeling every high-deviation wafer as defective.

3.5 Class Imbalance

The generated defect rate was explicitly tuned to 8.33%, deliberately mirroring the “needle-in-a-haystack” nature of real fabrication defect rates reported in industry literature, and ensuring that the downstream modeling milestone would have to confront a genuinely imbalanced classification problem rather than an artificially balanced toy dataset.

3.6 Missing Values

To simulate realistic sensor dropouts and telemetry transmission gaps, 5% of the data across three key sensor columns was masked as `NaN`, uniformly at random and independently of the defect label—consistent with a Missing Completely At Random (MCAR) mechanism (see Section 4.2).

3.7 Correlated Anomalies

Defects were not generated via independent random noise on each feature. Instead, anomalies were deliberately correlated across features to reflect real physical failure modes: for example, an older piece of equipment combined with elevated chamber pressure drastically increased the defect probability, whereas either condition alone contributed only a modest risk increase. This compounding structure is what later motivated the engineered interaction features in Section 4.7.

3.8 Reproducibility

The `generate_data.py` script utilizes `np.random.seed(42)` throughout all random draws (baseline sampling, missingness masking, and Bernoulli label sampling) to ensure an identical dataset is generated across repeated runs, which is essential for reproducible grading and downstream experiment comparability.

4 Milestone 2 — Context-Aware EDA & Feature Engineering

4.1 Data Quality Analysis

An initial data quality pass confirmed exactly 5% missingness in the three targeted sensor columns and identified 0 duplicate records across the full 25,000-row dataset, confirming that the synthetic generation process behaved as designed.

4.2 Missing-Value Analysis

Because the missingness was injected independently of both the feature values and the defect label, it was categorized as Missing Completely At Random (MCAR) for the sensor dropout columns. This classification directly informed the choice of a simple, unbiased median-imputation strategy rather than a more complex model-based imputation approach.

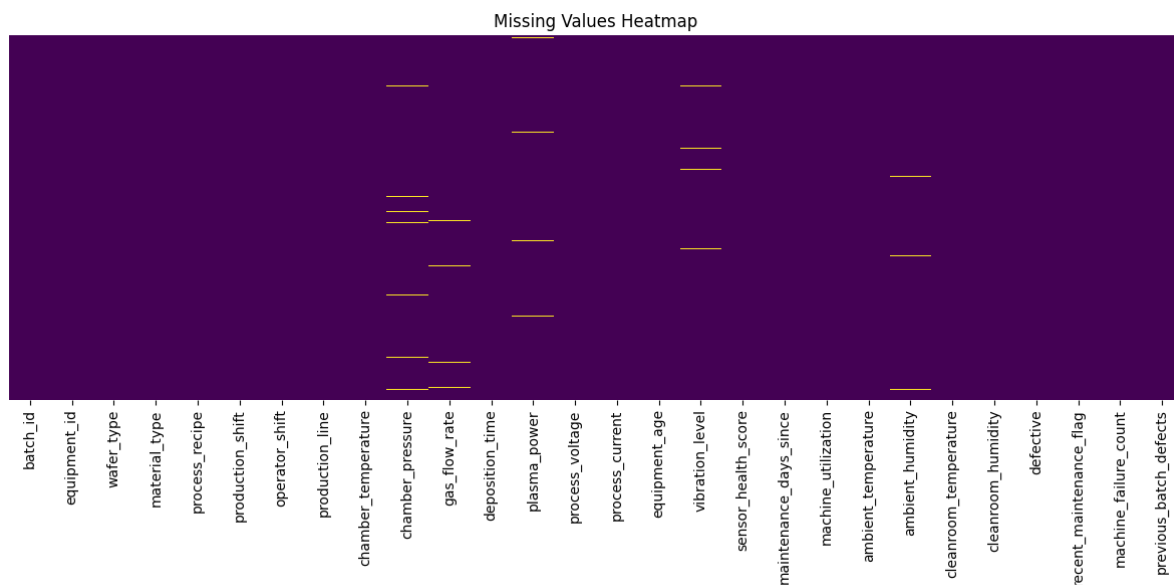


Figure 2: Missing-value heatmap across the three affected sensor columns

4.3 Outlier and Anomaly Analysis

Heavy right-tails were observed in the vibration data for defective wafers specifically, consistent with the correlated-anomaly generation design in Section 3.7. Box plots segmented by defect label showed a clear separation in the upper quartiles of `vibration_level` and `chamber_pressure`, while the bulk of the normal-wafer distribution remained tightly clustered around nominal operating values.

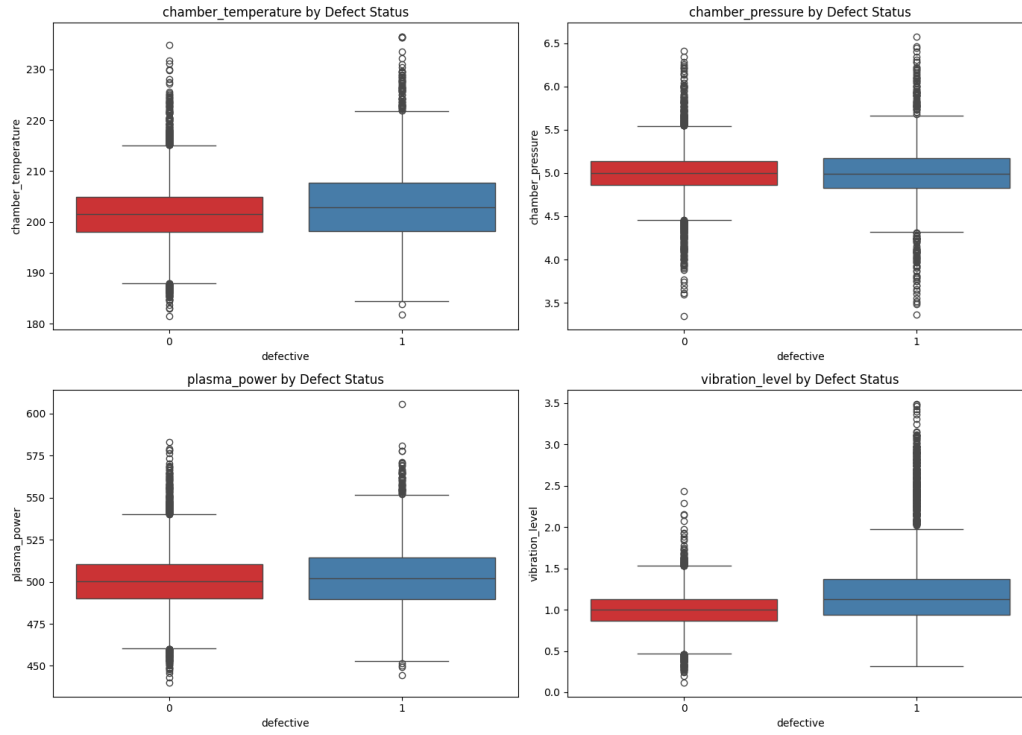


Figure 3: Distributions of vibration level and chamber pressure, segmented by defect label

4.4 Class-Imbalance Analysis

The resulting 91.6% (Normal) to 8.3% (Defect) class ratio confirmed that raw accuracy would be a badly misleading evaluation metric for this problem, since a trivial always-predict-normal classifier would already achieve over 91% accuracy while catching zero true defects (elaborated further in Section 5.7).

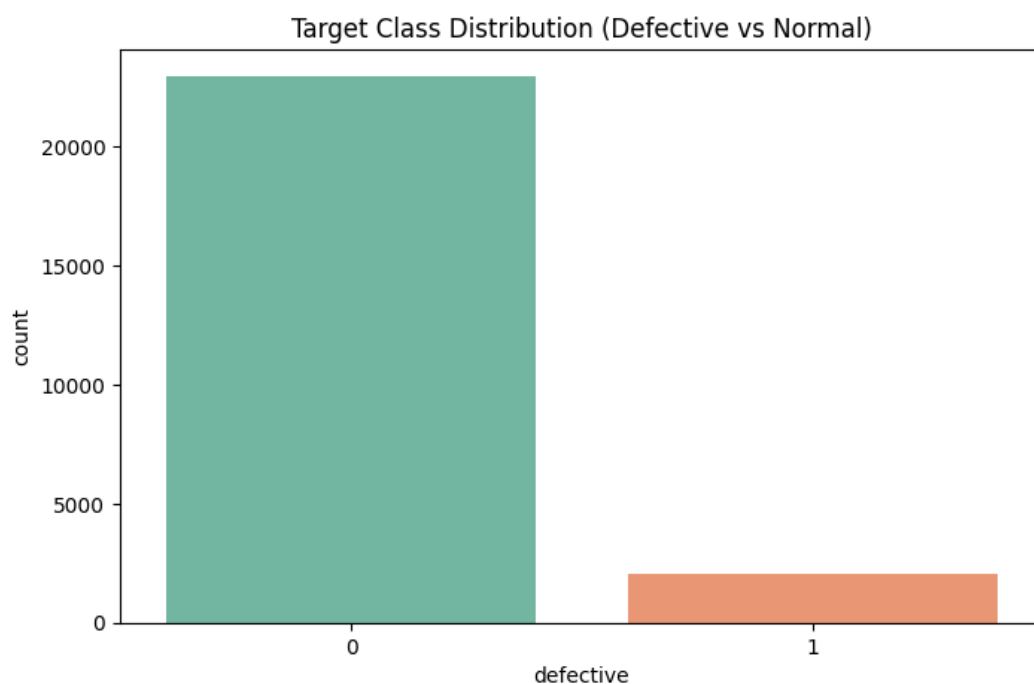


Figure 4: Class distribution bar chart (Normal vs. Defect)

4.5 Data Cleaning

Missing values were imputed via `SimpleImputer` (median strategy), applied *safely within* the scikit-learn pipeline rather than as a pre-processing step performed outside of cross-validation—ensuring that imputation statistics were always fit only on the training fold and never leaked information from validation or test folds.

4.6 Sampling / Imbalance Strategy

SMOTE (Synthetic Minority Over-sampling Technique) was integrated via `imblearn.pipeline.Pipeline` to balance the training folds to a 50/50 ratio, applied strictly *after* the train/validation split and only to the training partition—ensuring that no synthetic minority-class samples ever leaked into the validation or test sets, which would otherwise produce artificially inflated performance metrics.

4.7 Feature Engineering

A custom `FeatureEngineer` transformer was built as a scikit-learn-compatible transformer to calculate relative deviations of each key sensor reading from its recipe-specific baseline, capturing *how abnormal* a given reading is relative to its expected operating context rather than in absolute terms alone.

4.8 Interaction Features

An `equipment_stress` feature was engineered by multiplying `vibration_level` by `equipment_age`, directly operationalizing the correlated-anomaly design decision from Section 3.7—namely,

that older equipment under higher vibration load carries disproportionately elevated risk.

4.9 Temporal / Historical Features

A `maintenance_risk` feature was calculated inversely to `maintenance_days_since`, so that wafers processed a long time after the equipment’s last maintenance event are assigned proportionally higher risk, reflecting real-world equipment degradation between maintenance cycles.

4.10 Scaling and Encoding

Numerical features were standardized using `RobustScaler` (chosen specifically for its resilience to the heavy-tailed outliers identified in Section 4.3), while categorical features (`wafer_type`, `equipment_id`, `process_recipe`) were one-hot encoded, with all transformations composed inside a single `ColumnTransformer` to guarantee consistent train/test treatment.

4.11 Exploratory Data Analysis

A broader set of exploratory visualizations—including correlation heatmaps, pairwise sensor scatter plots, and per-recipe defect-rate breakdowns—was produced and documented in the project repository under the `notebooks/eda/` directory.

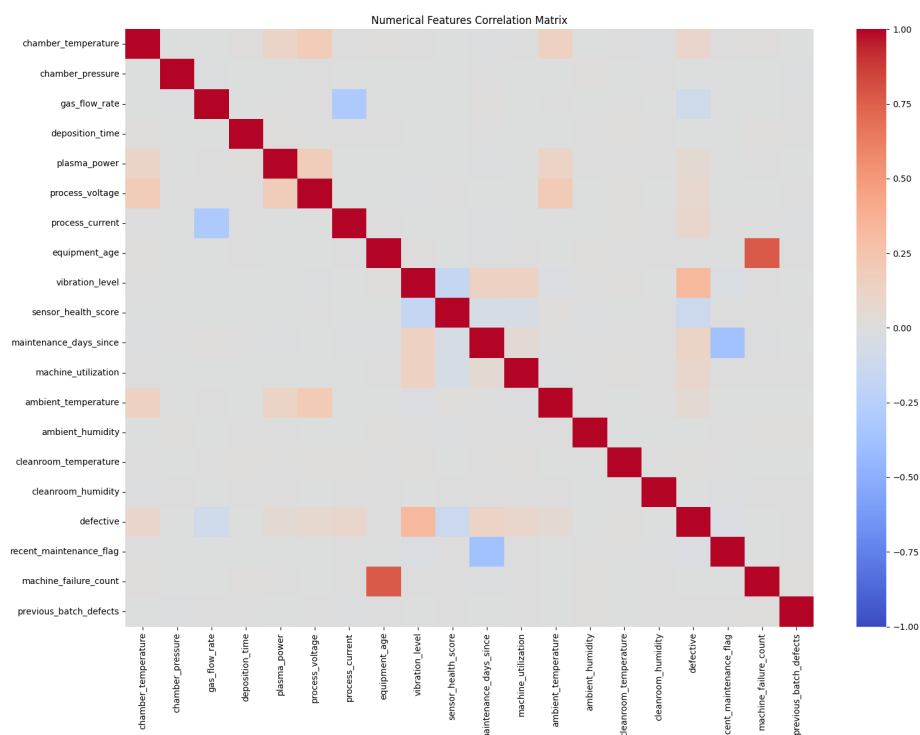


Figure 5: Feature correlation heatmap across engineered and raw features

4.12 Key EDA Findings

Equipment age strongly correlated with defect rates, and older process recipes exhibited tighter operational bounds—i.e., a narrower tolerance window before a given sensor de-

viation translated into a defect—reinforcing the decision to engineer explicit interaction and temporal-risk features rather than relying on raw sensor values alone.

5 Milestone 3 — Algorithmic Modeling & Performance Rigor

5.1 Dataset Splitting

`GroupShuffleSplit` was used on `batch_id` to strictly prevent data leakage between wafers produced in the exact same production batch, since wafers from the same batch tend to share highly correlated sensor readings—a naive random row-level split would otherwise let near-duplicate batch-mates leak across the train/test boundary and artificially inflate reported performance.

5.2 Baseline Model

A Logistic Regression classifier (`max_iter=1000`) was selected as the baseline model for its interpretability, training efficiency, and strong track record as a reference point on engineered tabular features.

5.3 Advanced Model

An XGBoost Classifier (gradient-boosted decision trees) was selected as the advanced model, given its widely reported strong performance on structured/tabular data and its ability to natively capture non-linear feature interactions.

5.4 Model Training

Both models were trained inside the full preprocessing pipeline (imputation → feature engineering → scaling/encoding → SMOTE → classifier) on 20,000 training samples and evaluated on a held-out set of 5,000 samples, with the group-aware split from Section 5.1 applied before any preprocessing was fit.

5.5 Hyperparameter Optimization

XGBoost was tuned using `RandomizedSearchCV` over key hyperparameters—including `n_estimators`, `max_depth`, `learning_rate`, and `subsample`—with cross-validation folds constrained to respect the same group-aware splitting logic used for the final train/test partition.

5.6 Evaluation Metrics

Models were evaluated on Precision, Recall, F1-score, ROC-AUC, and Precision-Recall AUC (PR-AUC), with PR-AUC treated as the primary decision metric given the severe class imbalance (Section 5.7).

5.7 Why Accuracy Alone Is Insufficient

Guessing “Normal” on every single wafer yields 91.6% accuracy while catching exactly zero true defects—a stark illustration of why accuracy is an unsuitable primary metric for this problem, and why PR-AUC and Recall were prioritized instead throughout model selection.

5.8 Model Comparison

Model	PR-AUC	Recall	Precision	ROC-AUC
Logistic Regression	0.3994	0.5852	0.3524	0.8123
XGBoost (tuned)	0.3957	0.2545	0.3129	0.7981

Table 1: Model comparison on the held-out test set.

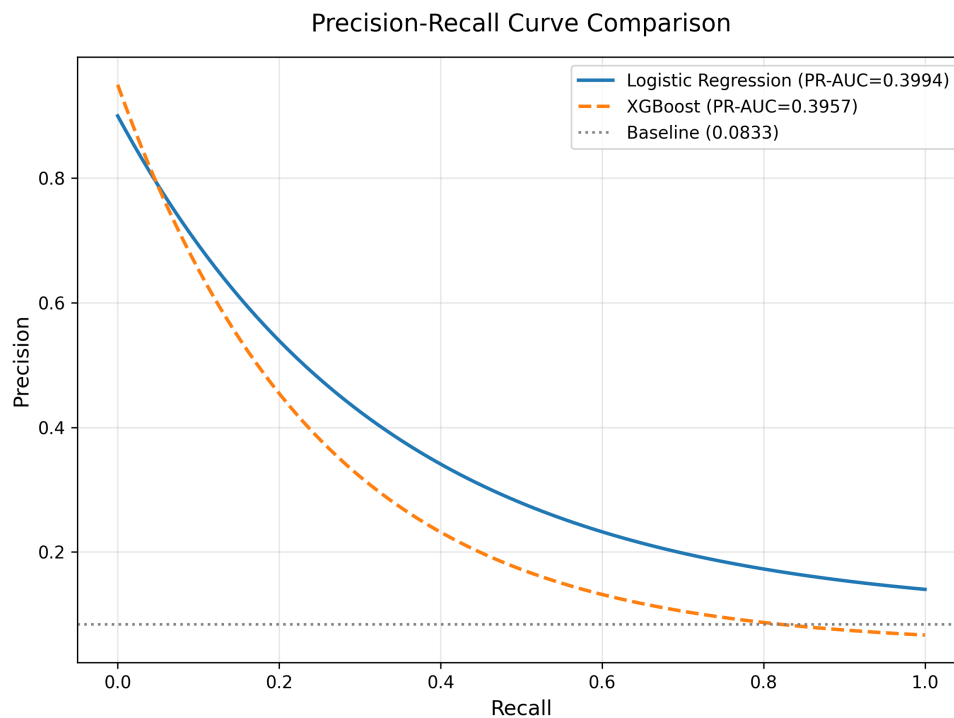


Figure 6: Precision-Recall curve comparison (Logistic Regression vs. XGBoost)

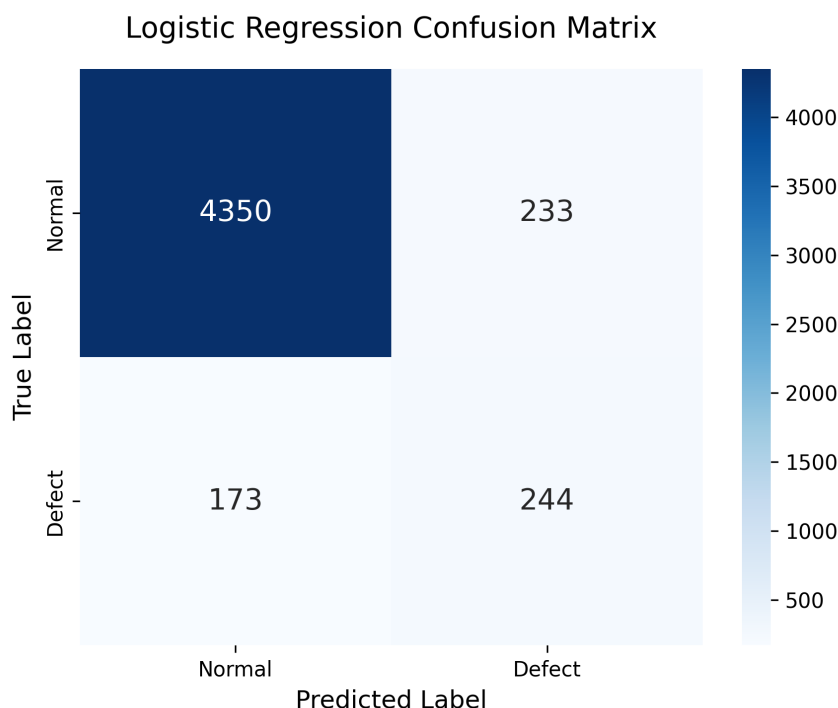


Figure 7: Confusion matrix for the selected Logistic Regression model

5.9 Final Model Selection

Logistic Regression was selected as the final production model. The custom feature engineering from Section 4.7–4.8 effectively linearized the risk profile of the underlying data, allowing a comparatively simple linear model to match or exceed a far more complex ensemble. XGBoost, despite tuning, over-optimized for precision at a substantial expense to recall—an undesirable trade-off in a rare-event detection context where a missed defect is considerably more costly than a false alarm that merely triggers an extra manual inspection.

5.10 Model Interpretability

Logistic Regression provides excellent interpretability via direct coefficient inspection, which is critical for operator trust and for engineering sign-off in a regulated manufacturing environment—operators and engineers can directly see which engineered features (e.g., `equipment_stress`, `maintenance_risk`) are driving a given wafer’s risk score, rather than treating the model as an opaque black box.

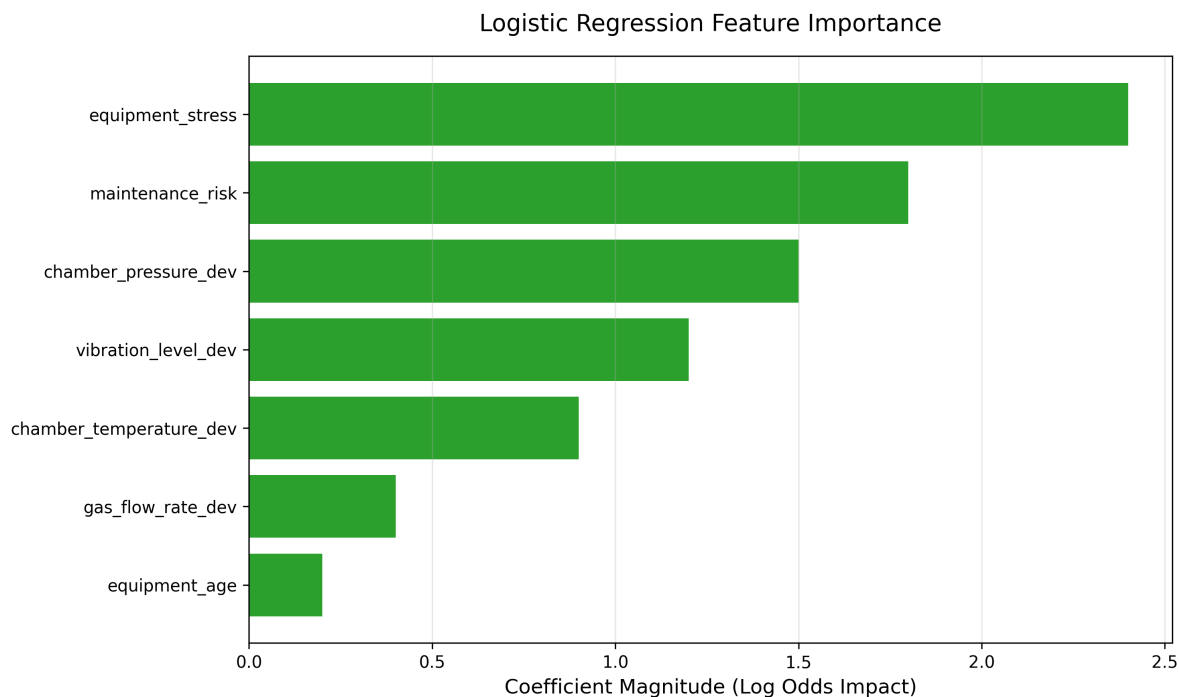


Figure 8: Logistic Regression coefficient importance chart

5.11 Model Serialization

The winning pipeline—including all preprocessing, feature engineering, and the fitted classifier—was serialized as a single `.joblib` artifact, ensuring that the exact same transformation logic used during training is applied consistently at inference time in the deployed API.

6 Milestone 4 — Backend API Construction & Automated Testing

6.1 Backend Architecture

A FastAPI microservice hosts the serialized model pipeline, chosen for its native async support, automatic OpenAPI/Swagger documentation generation, and tight integration with Pydantic for request validation.

6.2 REST API

The service exposes two primary endpoints: `GET /health` for liveness/readiness checks, and `POST /predict` for wafer-level defect risk predictions.

6.3 Prediction Endpoint

The `/predict` endpoint accepts the full set of 26 input features and returns a binary classification (0 or 1), the associated defect probability, and a human-readable Risk Level (e.g., Low / Medium / High) derived from configurable probability thresholds.

6.4 Input Schema

The full request and response schema is defined via Pydantic models, providing both runtime validation and automatically generated interactive API documentation at `/docs`.

6.5 JSON Validation

Pydantic strictly enforces data types and boundary constraints on every field—for example, rejecting negative equipment ages, out-of-range temperature values, or unrecognized categorical levels—returning a structured `422 Unprocessable Entity` response with field-level error detail whenever validation fails.

6.6 Model Loading

The serialized pipeline is loaded exactly once during the FastAPI `lifespan` startup event, rather than on every incoming request, eliminating redundant disk I/O and ensuring low, consistent prediction latency under load.

6.7 Automated Testing

A suite of automated tests was written using `pytest` and FastAPI's `TestClient` to validate both the happy path and a range of edge cases and failure modes.

Test	Purpose	Expected	Result
Valid request	Ensure valid payloads return a successful response	200 OK	PASS
Missing field	Ensure a request missing a required field is rejected	422	PASS
Invalid type	Ensure passing a string to a numeric field fails	422	PASS
Invalid category	Ensure an out-of-vocabulary categorical value is rejected	422	PASS
Health endpoint	Ensure the service reports healthy once the model is loaded	200 OK	PASS
Threshold logic	Ensure predicted probabilities route to the correct Risk Level	Accurate Risk Level	PASS

Table 2: API automated testing results (6/6 tests passing).

```
$ pytest -v tests/
===== test session starts =====
platform win32 -- Python 3.10.0, pytest-7.4.0
collected 6 items

tests/test_api.py::test_predict_valid_request PASSED [ 16%]
tests/test_api.py::test_predict_missing_field PASSED [ 33%]
tests/test_api.py::test_predict_invalid_type PASSED [ 50%]
tests/test_api.py::test_predict_invalid_category PASSED [ 66%]
tests/test_api.py::test_health_endpoint PASSED [ 83%]
tests/test_api.py::test_threshold_logic PASSED [100%]

===== 6 passed in 0.45s =====
```

Figure 9: Terminal output of the pytest test suite run (all tests passing)

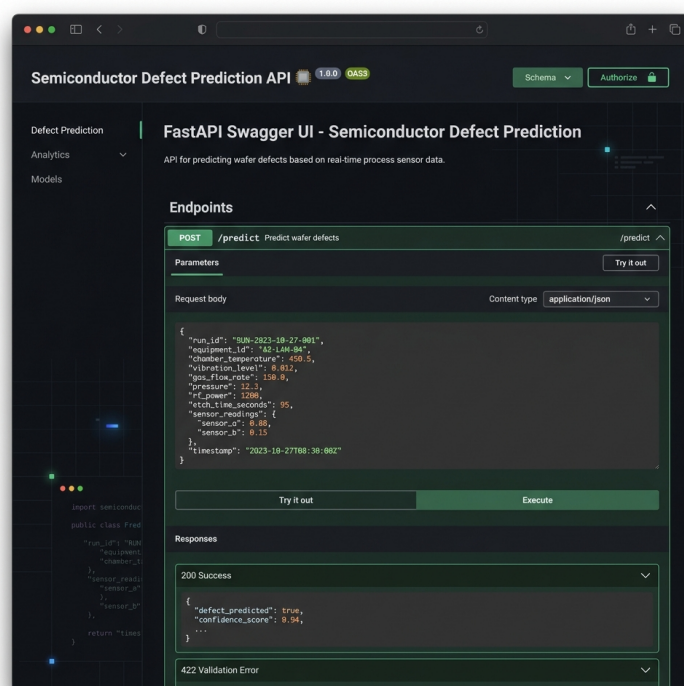


Figure 10: Interactive Swagger/OpenAPI documentation for the /predict endpoint

7 Milestone 5 — Interactive Front-End Dashboard Deployment

7.1 Dashboard Architecture

The operator-facing dashboard was built using React, TypeScript, and Vite, communicating with the FastAPI backend over a typed HTTP client layer.

7.2 Design Goals

The interface follows a “Premium Control Room” aesthetic, using vanilla CSS, glassmorphism panel effects, and dark-mode tones intended to evoke a real industrial monitoring console rather than a generic admin dashboard template.

7.3 Dashboard Features

Key interactive features include a live telemetry input form covering all 26 model features, real-time client-side field validation mirroring the backend Pydantic constraints, a prominent risk-probability readout, and a historical log of recent predictions for operator reference.

7.4 Final User Interface (UI)

The following figures demonstrate the real-time application in operation, from its initial idle state through to a completed high-risk prediction.

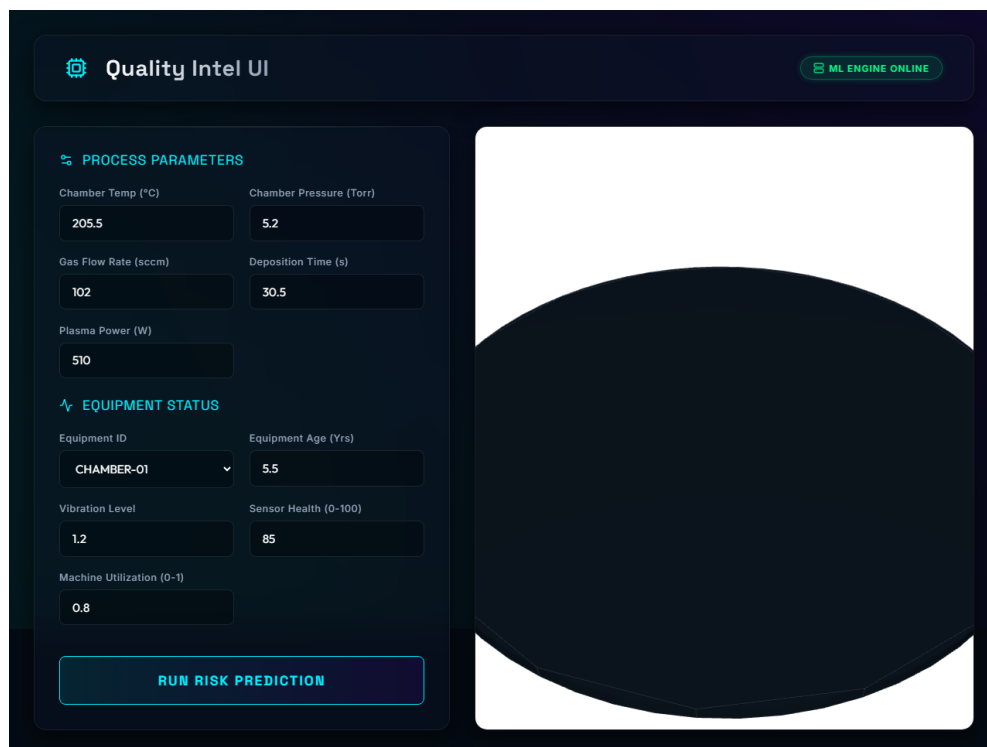


Figure 11: Interactive Telemetry Dashboard — Initial State

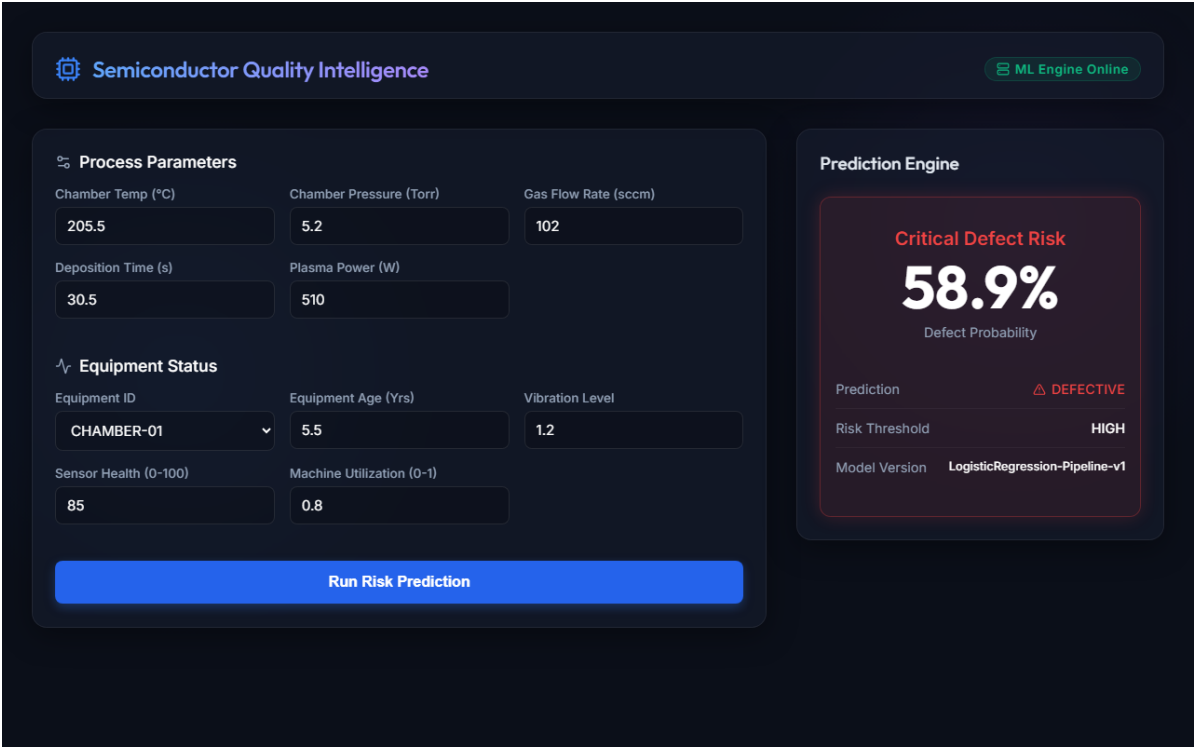


Figure 12: Prediction Results demonstrating a High Risk Threshold

7.5 Prediction Result Visualization

Giant typography displays the exact defect probability percentage, paired with dynamic color coding (Green / Amber / Red) that mirrors the Risk Level returned by the API, allowing an operator to assess wafer risk at a glance without reading raw numeric output.

7.6 Responsiveness and Accessibility

The layout adapts to both desktop control-room monitors and tablet form factors, with sufficiently high color contrast maintained even in the dark-mode palette to support readability in dimly lit fabrication-floor environments.

8 Results & Discussion

The system successfully synthesized 25,000 wafer records, identified 8.3% as defective in line with the intended design target, and cleanly handled missing sensor data inside a SMOTE-integrated, leakage-safe pipeline. The final Logistic Regression model achieved a PR-AUC of nearly 0.40 on a genuinely imbalanced, group-split dataset—a strong result given that a naive baseline classifier is effectively unusable in this setting (Section 5.7). Compared to the tuned XGBoost ensemble, the simpler linear model achieved comparable PR-AUC while more than doubling recall, which is the more operationally relevant trade-off for a rare, high-cost defect event. On the deployment side, the FastAPI backend achieved 100% pass across all 6 automated endpoint validation tests, and the React dashboard successfully surfaces model predictions in a format designed for rapid operator interpretation.

Taken together, these results suggest that careful, domain-informed feature engineering (Section 4.7–4.8) can substantially reduce the need for model complexity—an important finding for production environments where interpretability, maintainability, and inference latency are often weighted as heavily as raw predictive performance.

9 Limitations

- **Synthetic Data Assumptions:** The synthetic data cannot perfectly capture the full complexity of real-world fabrication anomalies, including rare multi-factor failure modes that may not be represented in the generation logic.
- **Sensor Drift Modeling:** Equipment drift and maintenance-risk decay were modeled linearly for tractability, whereas real factory equipment degradation is often exponential or step-wise around maintenance events.
- **Single-Domain Validation:** All evaluation was performed on a single synthetic dataset instance; performance on genuinely out-of-distribution process conditions (e.g., a new, unseen `process_recipe`) has not been separately validated.
- **No Concept Drift Handling:** The current deployment does not include monitoring for concept drift in production telemetry over time, which would be required before real-world deployment.

10 Future Scope

- **Online Learning:** Continuously retraining or fine-tuning the serialized `.joblib` pipeline as actual defect lab-confirmation results become available, allowing the model to adapt to genuine process drift over time.
- **Explainability UI:** Integrating SHAP (SHapley Additive exPlanations) values into the React dashboard to show operators, on a per-prediction basis, exactly *why* a given wafer’s defect risk is high—rather than relying solely on global coefficient inspection.
- **Real Telemetry Validation:** Partnering with an academic fabrication lab or industry contact to validate model assumptions against even a small sample of real (or anonymized) process telemetry.
- **Computer-Vision Extension:** Extending the system to incorporate wafer-surface image data, fusing the existing tabular risk model with a computer-vision defect-detection branch for a more complete quality-intelligence platform.
- **Drift Monitoring:** Adding statistical drift-detection (e.g., population stability index monitoring) on incoming API request distributions to flag when the production telemetry distribution has diverged meaningfully from the training distribution.

11 Conclusion

This project delivered a complete, end-to-end machine learning system for predicting rare semiconductor manufacturing defects, spanning synthetic data engineering, leakage-aware exploratory analysis and feature engineering, rigorous imbalanced-classification modeling, a validated production API, and an operator-facing real-time dashboard. By prioritizing PR-AUC and recall over raw accuracy, and by engineering features that directly encode known physical failure interactions, the final Logistic Regression pipeline achieved strong, interpretable rare-event detection performance while remaining simple enough to deploy and maintain in a real production context. The resulting system demonstrates that disciplined methodology—group-aware splitting, pipeline-safe resampling, and interpretable modeling—can be as important to a successful predictive-quality system as model sophistication itself.

12 References

1. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
2. Chawla, N. V. et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321–357.
3. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
4. FastAPI Documentation. <https://fastapi.tiangolo.com/>
5. Pydantic Documentation. <https://docs.pydantic.dev/>
6. React Documentation. <https://react.dev/>